Sovra Verify: an evidence-first consistency auditor for procurement documents

Track 2B — Apertus application. Six pages excluding references. All figures in this report are measured; sources are named inline and full records are listed under References.

---

1. Problem and users

A procurement review compares the same commercial terms across documents that were written at different times by different functions: a master agreement, the buyer's accounts-payable policy, and the invoices raised under them. The disagreement that matters is rarely a missing number — it is the same term stated two ways, and finding it means reading the same clause in three places.

Large language models are good at the reading and the judging. They are unreliable as the *source*. A model asked to quote a document will produce a quotation that reads plausibly and does not appear in the document, and a reviewer has no cheap way to tell the difference. That failure mode makes a model-generated answer worse than no answer: it converts a slow task into a slow task plus a verification task.

Sovra Verify separates the two roles. The model reads and judges; the evidence comes from canonical extracted text that the model cannot write to. The output is a set of **potential** inconsistencies, each carrying the passage, page and character offsets it rests on, for a human to review.

The primary user is a procurement or finance reviewer performing a consistency check before an approval. **The system is not a lawyer, not a compliance authority, and not an approver.** It reports inconsistencies in the corpus it was given. It does not check external truth, does not decide which document governs, and produces no legally operative result. Every screen that can produce a finding states this next to the controls that produce it.

Scope: text-native PDF and UTF-8 TXT, one project, up to ten documents, 10 MB per file, with an explicit extracted-text budget. Oversized, encrypted and unsupported files are rejected with a reason; a scanned PDF reports "OCR not implemented" rather than yielding empty evidence. DOCX, OCR, authentication, integrations and multi-project support are out of scope for this prototype.

---

2. Architecture and the role of Apertus

The application is a Python 3.12 FastAPI backend with a React and TypeScript dashboard built by Vite and served by the backend. PyMuPDF extracts text-native PDFs; SQLite stores documents, passages, claims, analyses, findings, provider runs and review history. The architecture diagram is in `docs/diagrams/architecture.png`.

```
upload → extraction → claim identification → scoped comparison → validated citations → review → export
```

Apertus is the reasoning layer. `swiss-ai/Apertus-v1.5-70B`, served by CSCS at `https://api.inference.cscs.ch/v1` over OpenAI-compatible Chat Completions, does two jobs:

1. **Claim identification.** For each batch of passages it returns typed claims — subject, predicate, value, unit, scope, effective date — with a nominated passage id and character offsets.
2. **Scoped comparison.** For one bounded claim pair it returns a relation (`consistent`, `potential_conflict`, `insufficient_evidence`, `unrelated`), a short reasoning sentence, cited spans and explicit limitations.

Nothing else is delegated to the model. It never authors a quote, never chooses a provider or a URL, never selects a file path, and never decides a relation for a pair it was not shown. Constrained decoding, the Responses API, tool calls and embedding endpoints are all treated as unverified capabilities and are not relied on; output is validated application-side with Pydantic.

Provider policy. CSCS is primary. The Hugging Face router is a backup that is **off by default**, because it cannot serve the default model: its only Apertus entry is `swiss-ai/Apertus-8B-Instruct-2509`, and the `swiss-ai/Apertus-v1.5-70B` mapping reports `status: "error"`. The 70B model is never silently replaced by an 8B

model; selecting an 8B model requires `SOVRA_ALLOW_OPTIONAL_8B=1`, and switching providers requires both `SOVRA_BACKUP_ENABLED=1` and `SOVRA_ALLOW_BACKUP_FALLBACK=1`. A provider change is recorded on the run with `is_backup = true` and `downgraded_from` set.

Failure is a first-class outcome. Every call writes a `ProviderRun` with provider, model, mode, latency, token usage, error class, HTTP status and validation errors. A run whose comparison calls all failed finishes as `failed`, not as a clean empty finding list, and says so in the output and the export. Partial runs are labelled `partial` with the failures attached.

---

3. The evidence model

The evidence model is the core contribution, and it is enforced by construction rather than by prompting.

Canonical text is the authority. Each document's extracted text is stored once, and `(start_offset, end_offset)` index into it. Critically, offsets are **per document**: every span is resolved against the canonical text of the document it came from. An early implementation joined canonical text per passage across the whole corpus, which meant a span from the second document was sliced out of the first document's text. A test that asserts every produced citation equals its own document's canonical slice caught this before any result was reported.

Quotes are derived, never authored. The model nominates a passage id and an offset pair. The application validates that the passage exists, that the offsets are ordered and in range, that the span overlaps the passage it claims to come from, that the derived quote is at least eight non-whitespace characters, and that it is a substring of the passage — then slices the quote from canonical text. A quote string supplied by the model is discarded on sight.

Unresolvable evidence is rejected, not softened. A `potential_conflict` with no validated citation is dropped, and the count of dropped conflicts is reported as a limitation. A claim the model attributes to a passage id that does not exist is stored as `unresolved` and is never paired.

Uncertainty is labelled, not scored. Unknown dates and scopes stay `unknown` rather than being guessed. No calibrated confidence percentage is emitted, because the system is not separately calibrated; uncertainty is expressed as explicit limitation strings. Review states are `pending` / `confirmed_for_review` / `dismissed` — never "resolved" or "approved".

Retrieval is bounded and its gaps are visible. Only cross-document claim pairs whose normalised predicate, unit, entity and effective date align become candidates. Pairs describing the same difference are deduplicated so one conflict is reported once. Coverage statistics — pairs considered, pairs after deduplication, pairs compared, pairs dropped by cap, and the top reasons pairs were skipped — are stored on the analysis, so a retrieval miss cannot hide. A differing known effective date is treated as an explanation rather than a conflict, and the number of pairs skipped for that reason is reported so the reader can audit the choice. These statements are made on screen, in the candidate-coverage notice, and they are verified where a reviewer would read them: `make ui-check` drives the running interface in a headless browser and asserts that a compared pair's judged outcome is stated, that a completed run reporting nothing says so, and that no footer marker is present in the rendered text or the DOM. Over the four-document demo corpus that run compared 11 pairs and the panel read "11 compared pair(s) were judged: 10 potential conflict, 1 consistent".

Imported documents are data. Prompt templates state that passage text is untrusted user data which cannot alter the task, the output format, the provider or the tools. No tool, network request, code execution or file access is ever derived from document text, and adversarial evaluation rows carrying injection payloads are treated as data under test.

---

4. Evaluation

Method, corpus, raw observations, the provider ledger and the limitations are in `docs/EVALUATION.md` and `track_2b/data/evaluation/RESULTS.json`. The corpus is 25 hand-labelled synthetic pairs across seven

categories, frozen before prompt tuning. Inference was real (`swiss-ai/Apertus-v1.5-70B` on CSCS); 41 provider calls, 0 failed.

Class	Support	Precision	Recall	F1
consistent	8	1.000	0.375	0.545
potential_conflict	7	0.545	0.857	0.667
insufficient_evidence	7	0.000	0.000	0.000
unrelated	3	0.000	0.000	0.000

Accuracy 0.36 (9 of 25). **Abstention rate 0.44** (11 of 25). **Failed requests 0.00**, counted separately from abstentions. **Resolved citation rate on conflicts 100 %** (12 of 12 reported conflicts carried at least one validated span). Latency per case: mean 5,542 ms, median 5,581 ms. Tokens per case: mean 1,502.

The single figure hides two different failures, so the evaluation now reports them separately: **14 of the 25 cases ever reached the model**, and on that subset accuracy was **0.6429** (9 of 14). Of the 10 cases whose expected relation is a negative (`unrelated` or `insufficient_evidence`), 8 produced no inconsistency — the absence of a false alarm, not evidence of correctness.

Two things are worth stating plainly.

Abstention, not error, dominates the count. Eleven cases were never compared, because the candidate filter did not consider the pair alignable, and four of those produced only one claim. Abstaining is safe — no unsupported claim reaches a reviewer — but it is not useful, and it is the largest single component of the error.

The citation guarantee held. Every conflict the system reported carried a resolved span, and the separate end-to-end demo run over four documents produced 25 claims, 10 compared pairs and **zero** citations that did not equal an exact slice of their own document's canonical text.

The five false positives are all the model's own judgement, and this was established by experiment rather than assumed. An earlier revision of this report attributed the `valid_exception` errors to the exception clause being dropped before the comparison. That was wrong. The comparison prompt carries both claim passages in full, and the model cited the exception sentence as its own evidence before judging against it: for `eval-018` the span it quoted reads "as an exception to Policy 4.1", and its reasoning still called the values incompatible. A revision of the prompts that additionally placed the surrounding statements of each claim's own document in front of the model, alongside an instruction to extract every distinct statement including stated exceptions, was then measured against the same frozen corpus: 41 calls, 0 failed, and **every case landed on the same side as before**. The changes are kept because a model should see the document it is judging, not because they raised the score. The `date_change` and `missing_context` errors share the shape the prompt already forbids: a conflict asserted from fragments whose subject, scope or effective date is absent.

This evaluation is small, synthetic and single-run. It does not establish real-world performance, and no such claim is made.

---

5. Deployment, cost and security

Verified mode: a local application calling CSCS-hosted inference. That is external-network inference. **A local web app calling CSCS is not air-gapped**, and Swiss inference residency does not establish residency of the whole application — the database, stored originals, exports and logs remain the operator's responsibility. CSCS documentation states that prompts and responses are not recorded by CSCS; that covers the inference call, not the application.

On-premise, full Swiss-cloud and air-gapped deployments are **not claimed**. A local OpenAI-compatible Apertus adapter exists but has never been pointed at real on-premise hardware, and no no-network verification was performed.

Costs are bounded, and no dollar figure is reported. No verified price list was available, so a cost per case would be an invented number. The account's $10 Hugging Face credit is user-reported and was not measured, and no paid dedicated endpoint was deployed. Work per analysis is bounded by `max_output_tokens`, a comparison cap, a chunk cap and a total extracted-text budget; retries are capped at two with backoff and only for transient errors.

Credentials. Read from the process environment first and `~/.hermes/.env` second, with only named keys extracted, never shell-sourced, never copied into the repository or an image layer, and masked in every log, argument and diagnostic. The Docker launcher passes the two application variables into the container **by name** so no secret travels through a file or an argument, and `compose.yaml` has no `env_file` because that would leak every unrelated secret in the file. Judges supply their own credentials; no developer key ships with the submission.

Residual risks. No API authentication exists — this is a single-user local prototype and must not be exposed to a network. Real sensitive documents should not be sent to any provider without authorisation and a data review.

---

6. Limitations and next work

Accuracy is not sufficient to trust the tool. 0.36 on a 25-case corpus is a weak result and is reported as measured.

The next work is the bottleneck the evaluation found, not new features:

1. **Candidate coverage.** Abstention dominates the error. The pairing filter needs to over-generate candidates and let the model abstain, rather than under-generate and never ask.
2. **Surface abstention.** "Considered and not reported" should be a visible state, so an empty result is distinguishable from a pair that was never compared. Today the metric records a miss where the interface arguably did the right thing.
3. **Same-document context.** Exception and date-change false positives need the clause that modifies a term, which usually sits in the same document as the term.
4. **Real, reviewed data.** An authorised deidentified historical corpus reviewed by a consenting domain expert, measuring false positives and omissions against that reviewer's judgement rather than against our own labels. No customer, ROI or deployment claim exists today and none should be made until such a review happens.
5. **DOCX and OCR**, after a licensing review, and a **tested** on-premise or air-gapped Apertus deployment, which is currently an untested adapter.

What is deliberately not claimed: air-gapped operation, production readiness, legal or compliance conclusions, automated policy changes, dollar cost, customer adoption, or accuracy on real documents.

References

1. `docs/PROVIDER_VERIFICATION.md` — recorded CSCS and Hugging Face verification: endpoint responses, model ids, one real completion each, masked credentials.
2. `track_2b/data/evaluation/RESULTS.json` — measured evaluation output: confusion matrix, per-class metrics, per-case observations, provider ledger, run status.
3. `docs/EVALUATION.md` — evaluation method, corpus provenance, counts, failures and limitations.
4. `track_2b/data/evaluation/LABELS.md` — labelling rules and provenance for the frozen corpus.
5. `docs/PROGRESS.md` — append-only checkpoint log with exact commands, results, failures and fixes.
6. `docs/DEPLOYMENT.md` — deployment modes, credential injection, cost and security notes.
7. `track_2b/backend/sovra/citations.py`, `normalize.py`, `compare.py`, `pipeline.py` — the evidence model, pairing rules and failure handling as implemented.
8. `track_2b/tests/` — 248 unit, integration and end-to-end tests, including the cross-document citation assertion, tests that prove both credential audits detect a planted canary, and tests that read the comparison prompt that was actually sent to check the document context reaches the model.
9. CSCS LLM Inference API service documentation, `https://docs.cscs.ch/services/inference/api/#llm-inference-api-service`, consulted 2026-10-12; cross-checked against the live calls in reference 1.
10. `docs/diagrams/architecture.html` — architecture diagram source (diagram-design), exported to `architecture.svg` and `architecture.png`.
11. `track_2b/data/evaluation/PROOF.json` — the delivered demonstration run's evidence bundle: document hashes, provider ledger, and every citation offset re-derived from canonical text.
12. `scripts/audit_secrets.py` — the release gate over the publishable tree and any archive built from it; `make audit` runs it.